

Access Control Sui Audit



June 2, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Access Control	5
Delayed and Two-Step Ownership Transfer	7
Security Model and Trust Assumptions	8
Notes & Additional Information	11
N-01 Event-Related Improvements	11
N-02 Auth<Role> Can Outlast Role Membership Within a PTB	12
N-03 Multiple Improvements for the access_control Module	12
N-04 Inconsistent Foreign-Role Enforcement Across Read-Path Functions	13
N-05 Elapsed Increases to default_admin_delay_ms Can Be Bypassed When Scheduling New Admin Transfers	14
N-06 Self-Transfer of the Root Role Emits No-Op Events	15
Conclusion	16
Appendix	17
Issue Classification	17

Summary

Type	Library	Total Issues	6 (5 resolved)
Timeline	From 2026-05-21 To 2026-05-25	Critical Severity Issues	0 (0 resolved)
Languages	Move	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	0 (0 resolved)
		Notes & Additional Information	6 (5 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin audited the [OpenZeppelin/contracts-sui](#) repository. The `access_control` module was audited in full at commit [7f7a300](#), while the `delayed` and `two_step` ownership transfer modules were audited as a diff covering the changes between base commit [9b5b807](#) and target commit [7f7a300](#).

In scope were the following files:

```
contracts
├── access
│   └── sources
│       ├── ownership_transfer
│       │   ├── delayed.move
│       │   └── two_step.move
│       └── access_control.move
```

System Overview

The scope covers the access-related building blocks of the `contracts-sui` library: a role-based access control registry and two reusable ownership transfer wrappers. Together, they provide the foundation for governance and asset-handover patterns that downstream Sui Move protocols can compose into their own modules.

Access Control

The access control module implements role-based access control for Sui Move protocols. The library exposes a singleton `AccessControl<RootRole>` registry that consuming protocols instantiate at first publish and use as the single source of truth for who can execute privileged actions. Authorization is propagated through typed `Auth<Role>` witnesses, which act as self-validating proofs of role membership that gated functions can consume by reference without performing any further authorization checks in their body.

Registry singleton: The `new` constructor accepts only a One-Time Witness as `RootRole`, relying on the Sui VM guarantee that exactly one OTW value of a given type is produced when its defining package is first published. This constrains each module to at most one `AccessControl` of its own type, ever. A package upgrade that adds a new module does not produce a fresh OTW for that module, so protocols wishing to adopt the library after the fact must publish a new package whose `init` initializes its own registry rather than adding a module to an existing package.

Home-module restriction: Every mutating role entry point and the auth-mint path require the role type to be declared in the same module as the root role. The same-module check uses `type_name::with_original_ids` inside `assert_home_module`, which compares the package's original publish address rather than the current upgraded version. Roles introduced in later upgrades to that same module remain valid, while roles defined in unrelated modules or packages are rejected at the boundary. Together with the singleton invariant, this guarantees that every `Auth<Role>` in circulation was minted by the unique registry that owns `Role`'s home module, against an address that actually held the role.

Role management: Roles are stored in a `Bag` keyed by `TypeName` and lazily registered on first grant or admin reassignment. Each role tracks its membership set and the admin role authorized to manage it. By default, a newly registered role is administered by the root role,

and admin authority can be delegated to any other home-module role. The zero address is rejected as a grant target or transfer recipient, since it has no signing key and would render the role or registry unmanageable. Voluntary renouncement of an arbitrary role is allowed only for the caller's own account, ensuring no role admin can use the renounce path as a backdoor to revoke another holder while bypassing the standard revoke path. The role-management surface consists of:

- `grant_role`: grant a role to an account; caller must hold the admin role of the target role.
- `revoke_role`: revoke a role from an account; caller must hold the admin role of the target role.
- `renounce_role`: voluntarily relinquish a role for the caller's own account; blocked for the root role.
- `set_role_admin`: delegate management of a role to a different admin role; caller must hold the current admin role.
- `has_role`, `assert_has_role`, and `get_role_admin`: read-side queries for membership and admin assignments.
- `new_auth`: mint an `Auth<Role>` capability for the caller, scoped to the current PTB by the `drop`-only ability set.

Root role lifecycle: The root role is protected against direct grant, revoke, renounce, or admin reassignment. Changes to its holder are channeled through a timelocked flow that shares a single pending slot between transfer and renounce, so scheduling one overwrites the other. Transfer is pull-based: the prospective new admin calls `accept` themselves, which atomically removes the previous holder and grants the role to the caller. The renounce path leaves the registry permanently unmanaged and is one-way by design. The root-role lifecycle exposes:

- `begin_default_admin_transfer`: schedule a transfer to a non-zero target address; populates the pending slot.
- `accept_default_admin_transfer`: finalize the transfer once `default_admin_delay_ms` has elapsed; callable only by the proposed new admin.
- `begin_default_admin_renounce`: schedule a renounce; populates the same pending slot.
- `accept_default_admin_renounce`: finalize the renounce once the delay has elapsed; callable only by the current root holder.
- `cancel_default_admin_transfer`: clear either kind of pending action while it remains pending; callable only by the current root holder.

Delay reconfiguration: The `default_admin_delay_ms` value itself is mutable through a dedicated delayed flow. Decreases must wait the freed time (current minus new), preventing

the admin from instantly weakening the protection level by shortening the delay. Increases wait `min(new_delay_ms, MAX_DELAY_INCREASE_WAIT_MS)`, capped at 48 hours so that scheduling a long delay does not require waiting the full new value before it takes effect. An upper bound of 60 days applies to any configured delay. A pending transfer or renounce is unaffected by a subsequently accepted delay change, since each pending action locks in the timelock under which it was originally scheduled. The finalization step is permissionless, because the schedule itself was committed by the root admin at the begin step. The delay-reconfiguration surface is:

- `begin_default_admin_delay_change`: schedule a new delay value with the computed wait.
- `accept_default_admin_delay_change`: apply the pending delay once its schedule has elapsed; permissionless.
- `cancel_default_admin_delay_change`: clear a pending delay change; callable only by the current root holder.

Delayed and Two-Step Ownership Transfer

The `delayed_transfer` and `two_step_transfer` modules under `ownership_transfer` provide reusable wrappers that gate handover of arbitrary `key + store` objects, the first behind a configurable minimum delay and the second behind a pull-based acceptance step. The diff under review covers only non-functional rearrangements:

- Section comment headers (`Errors`, `Structs`, `Public Functions`, `Test-Only Helpers`) were inserted to group declarations.
- The `WrappedKey` struct was relocated under the new `Structs` header.
- The calls to `option::fill(&mut self.pending, ...)` in `schedule_transfer` and `schedule_unwrap` were rewritten as `self.pending.fill(...)` using method-call syntax.
- A redundant `new_owner: new_owner` field shorthand in an emitted event was collapsed.

Observable behavior, parameter shapes, error codes, access checks, and storage layout are unchanged by the diff.

Security Model and Trust Assumptions

The security of the access control module rests on Sui's One-Time Witness guarantee, the home-module rule, and the correctness of the consumer protocol's initial role configuration. The registry does not opine on how roles map to off-chain identities; protocols inherit the security of the addresses they choose to grant root and per-role admin authority to. The following system-level assumptions apply:

- **One registry per consumer module.** Each `AccessControl<RootRole>` is the unique registry for its home module, established when the consumer module's `init` function calls `new` on first publish. Adopting the module in a later upgrade of an already-published package is not supported.
- **Home-module rule.** Only role types declared in the same package and module as `RootRole` can be granted, revoked, renounced, or minted as `Auth<Role>`, enforced by `assert_home_module`.
- **Timelocked root-role changes.** Every change to the root-role holder and to `default_admin_delay_ms` is gated on a delay chosen by the root admin and capped at 60 days; off-chain monitors rely on this window to detect and react to scheduled actions.
- **PTB-scoped authorization.** `Auth<Role>` certifies that the bound address held `Role` when `new_auth` ran in the current PTB; the absence of `store`, `key`, and `copy` confines it to that PTB.

Root role holder (default admin): The address holding the root role of a consumer protocol controls the registry. The root role is self-administering and cannot be granted or revoked outside the timelocked flow. Consumers must trust the root holder to operate within the configured delay window: while the holder cannot bypass the timelock for transfer, renounce, or delay decrease, they retain unrestricted authority within those bounds. A compromised root holder keeps the ability to schedule a transfer to an attacker, but the configured delay defines the recovery window during which the current root, or an honest counter-action, can cancel the pending action. The choice of `default_admin_delay_ms` therefore directly determines the

response time within which the protocol must monitor on-chain state for malicious scheduling. The root holder is authorized to:

- Grant and revoke every non-root role for which it remains the configured admin role via `grant_role` and `revoke_role`.
- Reassign the admin role of any non-root role via `set_role_admin`.
- Schedule and cancel delay reconfigurations via `begin_default_admin_delay_change` and `cancel_default_admin_delay_change`.
- Schedule, cancel, or finalize its own transfer via `begin_default_admin_transfer` and `cancel_default_admin_transfer`, or its renounce via `begin_default_admin_renounce` and `accept_default_admin_renounce`.

Delegated role admins: When `set_role_admin` is invoked, the chosen admin role gains the authority to manage membership of the target role. Consumers must trust each role admin to govern the roles under its control. Because the root role can reassign any non-root role's admin relationship at any time, role admins are ultimately subordinate to the root holder. A delegated role admin is authorized to:

- Grant the administered role to additional accounts via `grant_role`.
- Revoke the administered role from any account via `revoke_role`.
- Reassign the administered role's admin to a different role via `set_role_admin`, implicitly handing off control.

Role holders: A holder of a non-root role can exercise whichever privileged actions the consuming module gates on that role and cannot grant or revoke for others unless they also hold the relevant admin role. The typed `Auth<Role>` capability is `drop`-only and bound to the sender at mint time, so authorization captured in a programmable transaction does not persist beyond it. A role holder is authorized to:

- Voluntarily relinquish membership for their own account via `renounce_role`.
- Mint a PTB-scoped authorization witness via `new_auth` and pass `&Auth<Role>` to functions in the consumer module gated on that role.

Pending root admin: When a transfer is scheduled, the proposed new admin must be a non-zero address but is otherwise unconstrained and does not need to have any prior relationship with the registry. Acceptance is pull-based, so an unwilling or unreachable recipient simply leaves the transfer pending until the current root holder cancels or overwrites it. The proposed admin gains the root role only by explicitly calling `accept_default_admin_transfer` after the delay, at which point the previous holder is revoked atomically with the grant.

Integrating modules: The library does not enforce a particular gating discipline on consuming code. Integrators are responsible for:

- Taking `&Auth<Role>` in functions they intend to protect, since the library does not inspect the consumer's call sites.
- Declaring every role they need in the same module as their root role, so the home-module check in `assert_home_module` accepts them.
- Choosing an appropriate `default_admin_delay_ms` at construction time via `new`, since the value can only be tightened or relaxed through the delay-reconfiguration flow afterward.

The singleton invariant is enforced by the OTW check inside `new`, but only for modules whose `AccessControl` is routed through `new` in the package's first-publish `init`. Modules added in later upgrades to the same package cannot adopt the library and must instead live in a separate package with its own initializing module.

Notes & Additional Information

N-01 Event-Related Improvements

Throughout the [access_control module](#), multiple opportunities to improve event coverage and consistency were identified:

- **Silent overwrite of pending slots:** The [begin_default_admin_transfer](#), [begin_default_admin_renounce](#), and [begin_default_admin_delay_change](#) functions overwrite any existing pending operation without first emitting the corresponding [DefaultAdminTransferCancelled](#) or [DefaultAdminDelayChangeCancelled](#) event. As a result, an off-chain monitor that observes two consecutive [DefaultAdminTransferScheduled](#) events must infer that the first was implicitly cancelled. The Solidity [_setPendingDefaultAdmin](#) and [_setPendingDelay](#) helpers explicitly emit the cancel event on overwrite, giving indexers an unambiguous signal. Consider doing the same in the Move implementation.
- **Inconsistent [sender](#) field across events:** The [RoleGranted](#) and [RoleRevoked](#) events include a [sender](#) field identifying the transaction sender who triggered the change, whereas [RoleAdminChanged](#), [DefaultAdminTransferScheduled](#), [DefaultAdminRenounceScheduled](#), [DefaultAdminTransferCancelled](#), [DefaultAdminDelayChangeScheduled](#), and [DefaultAdminDelayChangeCancelled](#) do not. Consider adding a [sender](#) field to the remaining events for uniform off-chain attribution.

Consider applying the above improvements to give off-chain monitors and indexers a more complete and consistent view of the registry's state transitions.

Update: Resolved in pull requests [#338](#) and [#339](#).

N-02 `Auth<Role>` Can Outlast Role Membership Within a PTB

The `new_auth_function` verifies role membership once at mint time and returns an `Auth<Role>` carrying only the sender's address. It has no version or liveness pointer back into the registry. The struct has `drop` but lacks `store`, `key`, and `copy`, so the capability is confined to a single PTB.

Because functions gated by `&Auth<Role>` have no way to recheck the registry, the sender can mint `Auth<R>`, then revoke or renounce `R` for themselves, and still invoke an action gated by `&Auth<R>` later in the same transaction. Within that PTB, `has_role(sender, R)` returns `false` while `&Auth<R>` remains a valid authorization proof, so the protocol's two views of role membership disagree. This is the intended capability-based design, but consumer protocols that treat `Auth<Role>` as a live view of membership may build incorrect invariants on top of it.

Consider expanding the documentation of the `Auth<Role> struct` to clearly state that it is a snapshot taken at mint time, not a live view of role membership. Holding `&Auth<Role>` proves only that the bound address held `Role` at some point in the current PTB, not that they still do at the gated call site.

Update: Resolved in [pull request #336](#).

N-03 Multiple Improvements for the `access_control` Module

Throughout the `access_control module`, multiple opportunities to improve type safety and ergonomics were identified:

- **Dedicated storage for the root admin:** The root role has at most one holder, yet the `AccessControl struct` stores its membership inside the shared `roles Bag` as a `RoleData` entry whose `members VecSet` is always a singleton. Consider replacing it with a dedicated `Option<address>` field on the struct to express the singleton constraint directly and simplify the transfer and renounce paths.
- **Explicit initial admin in the constructor:** The `new function` implicitly grants the root role to `ctx.sender()`, while the Solidity `AccessControlDefaultAdminRules constructor` takes the initial default admin as a parameter. Consider mirroring this

signature so the publisher can seed the role with a multisig or governance address at construction time.

- **Typed role storage via `Table` instead of `Bag`**: The `roles` field is declared as a heterogeneous `Bag`, even though every value it stores is statically known to be a `RoleData`. Consider replacing it with a `Table<TypeName, RoleData>` to lift the value type into the static type system and rule out by construction any insertion of a foreign value type.
- **Redundant `account` parameter in `renounce_role`**: The `renounce_role` function accepts an `account: address` parameter that is immediately constrained to equal `ctx.sender()` via the `ECannotRenounceForOtherAccount` check, leaving the argument with no degrees of freedom. The signature mirrors OpenZeppelin's Solidity implementation, where the explicit parameter is meaningful because meta-transaction forwarders (for example, ERC-2771) can decouple the effective sender from `msg.sender`. However, Sui Move has no equivalent forwarder pattern, and `ctx.sender()` is the only available authority. Consider removing the parameter and reading `ctx.sender()` directly inside the function body to simplify the call site.

Consider applying the above improvements to tighten the type system around the role registry, and remove parameters that carry no semantic effect under Sui Move's authority model.

***Update:** Resolved in [pull request #337](#).*

N-04 Inconsistent Foreign-Role Enforcement Across Read-Path Functions

The `AccessControl<RootRole>` registry enforces that every queried `Role` type must be declared in the same module as `RootRole`. Mutation entrypoints such as `grant_role` consistently call `assert_home_module` and abort with `EForeignRole` when a foreign type is supplied, but the read-path is inconsistent: `get_role_admin` aborts with `EForeignRole`, while `has_role` returns `false` and `assert_has_role`, which forwards directly to `has_role`, aborts with `EUnauthorized`. The mismatch yields misleading diagnostics, since a foreign `Role` passed to `assert_has_role` surfaces as a permission failure rather than a type-scoping violation, and `has_role` returns a falsy value for a type that can never be a valid member, conflating "role not held" with "role cannot exist in this registry".

Consider aligning the three read-path functions on a single convention: invoke `assert_home_module` from `has_role` so foreign types abort with `EForeignRole` consistently with the mutation entrypoints and `get_role_admin`.

Update: Acknowledged, not resolved. The OpenZeppelin development team stated:

`_has_role` is a permissive membership predicate: for any role type that is not present in the registry, including a foreign role, returning false is semantically correct. The caller does not hold that role in this registry. Adding `assert_home_module` to `has_role` would impose extra type-checking gas on every valid membership query, while only improving diagnostics for invalid foreign-role inputs. Since mutation paths and capability minting already enforce `EForeignRole`, this does not create an authorization or state-safety issue._

N-05 Elapsed Increases to `default_admin_delay_ms` Can Be Bypassed When Scheduling New Admin Transfers

The `access_control` module gates root role transfers and renounces behind a configurable timelock, `default_admin_delay_ms`. Changes to the delay itself are also scheduled: `begin_default_admin_delay_change` records a `PendingDelayChange` together with a `schedule_after_ms` timestamp, and `accept_default_admin_delay_change` writes the new value into storage once that schedule has elapsed. The accept function is permissionless, so any account may call it.

However, until `accept_default_admin_delay_change` actually runs, `default_admin_delay_ms` still holds the previous value. The `begin_default_admin_transfer` and `begin_default_admin_renounce` functions read that field directly when computing `execute_after_ms`. Since Sui transactions can bundle multiple Move calls into a single programmable transaction block, the root admin can place a `begin_default_admin_transfer` (or `begin_default_admin_renounce`) call ahead of `accept_default_admin_delay_change` within the same PTB. When a delay increase is the pending change, the transfer then locks in `execute_after_ms` under the previous, shorter delay, even though its schedule has elapsed and the longer delay is about to take effect. Off-chain monitors that observed the `DefaultAdminDelayChangeScheduled` event and waited out the schedule before relaxing their watch can therefore be caught off guard, since the protocol-promised longer delay never applied to the transfer the admin scheduled.

The Solidity `AccessControlDefaultAdminRules` reference avoids this exposure by exposing the effective delay through a `defaultAdminDelay` getter that returns the pending value as soon as its schedule has passed. Every begin function routes through that getter rather than through raw storage.

Consider removing the `accept_default_admin_delay_change` step entirely and applying the pending delay implicitly once its schedule has elapsed. This can be achieved by routing every internal read of `default_admin_delay_ms` through a private getter that returns the pending value when `clock.timestamp_ms()` is at or past `schedule_after_ms`, and clearing the pending slot opportunistically on the next mutation.

Update: Resolved in [pull request #338](#).

N-06 Self-Transfer of the Root Role Emits No-Op Events

`accept_default_admin_transfer` unconditionally revokes the prior root holder and grants the new admin, even when both addresses are equal. If a root holder schedules a transfer to themselves and accepts it after the delay, the function emits `RoleRevoked` and `RoleGranted` for the same address with no net state change. The behavior diverges from `grant_role` and `revoke_role`, which short-circuit and skip event emission when the requested membership change is a no-op, and may cause off-chain consumers tracking root-role history to log a false admin handover that never actually occurred.

Consider rejecting the self-transfer at the schedule step by asserting that `new_admin` differs from the current root holder inside `begin_default_admin_transfer`, which prevents the no-op from consuming the timelock and accept ceremony and surfaces the mistake immediately to the caller.

Update: Resolved in [pull request #335](#).

Conclusion

The access control package implements role-based access control for Sui Move protocols, exposing a singleton `AccessControl<RootRole>` registry whose authority is propagated to consumer code through typed `Auth<Role>` witnesses, alongside two reusable ownership transfer wrappers that gate handover of arbitrary `key + store` objects. OpenZeppelin audited the `access_control` module in full and reviewed the changes introduced between the two referenced commits for the `delayed` and `two_step` ownership transfer modules. The rest of the codebase was outside the scope of this engagement.

The in-scope code is confined to a small set of Move modules and follows Sui-native patterns, including a one-time-witness-anchored singleton registry, type-name-based home-module enforcement for every role-typed entry point, and a timelocked transfer-or-renounce flow modeled after the equivalent EVM reference implementation. Only six notes were raised, with no critical-, high-, medium-, or low-severity findings within the audited scope.

The codebase separates the access control core from the ownership-transfer helpers and is accompanied by a test suite covering the in-scope modules. The diff under review for `delayed.move` and `two_step.move` is limited to non-functional rearrangements and leaves observable behavior, parameter shapes, and access checks unchanged. Addressing the items raised in the notes would tighten the library's conventions and improve internal consistency.

The OpenZeppelin development team is thanked for their responsiveness and willingness to provide technical context throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.